课程编号：3150550011036　　课程性质：必修

导航电子地图

课程实验报告

学院：测绘学院

专业：导航工程

地点：武汉大学

班级：测绘六班

姓名：刘峻琰

学号：2023302141249

教师：唐健

2026 年 3 月 2 日 至 2026 年 6 月 6 日

目 录

一、 概述 3
1.1 实验背景 3
1.2 研究现状 3
1.3 实验内容 4
二、 实验原理 5
2.1 实时定位 5
2.2 导航坐标系及转换 5
2.3 路径规划 7
三、 实验设计 9
3.1 实验框架 9
3.2 xml 页面设计 11
3.3 模块设计 13
3.3.1 应用主活动设计 13
3.3.2 设置页面设计 14
3.3.3 NEMA 工具类设计 14
3.3.4 异步路由与空间解析核心类设计 15
四、 关键技术及解决方案 16
4.1 地图加载与选点 16
4.2 定位功能 17
4.3 路径规划 18
4.4 导航功能 19
4.5 设置功能 21
4.6 自定义起点/终点图标与像素缩放 22
4.7 清空与路径管理 22
五、 实验结果 23
5.1 服务器端操作 23
5.2 基本功能 24
5.3 拓展功能 26
六、 总结 28

一、 概述

1.1 实验背景

随着移动互联网技术、智能终端传感器硬件以及地理信息系统（GIS）的深度融合，位置服务（Location-Based Services, LBS）已深入渗透至社会生产与公众生活的各个角落。电子地图与移动端导航作为 LBS 的核心载体，其应用场景正逐步从传统的宏观城市道路网向微观的园区、校园等细粒度高精度场景延伸。

高校校园作为典型的大型复杂微观园区，集中了高密度的教学楼、科研实验区、生活宿舍区、文体场馆以及复杂的内部路网体系（包含车行道、人行步道、林荫台阶等）。以武汉大学为例，其校园依珞珈山而建，地形起伏显著，不同学部（如文理学部、信息学部、工学部等）之间的道路纵横交错，且存在大量非机动车道与行人专用通道。市面上的通用商用地图服务（如高德地图、百度地图）主要服务于城市级宏观交通路网规划，受限于数据更新周期与采集成本，对于校园内部的细粒度道路、特定建筑物（如“信部食堂”、“信息学部图书馆”、“测绘学院”等）的覆盖精度、命名规范及拓扑联通性往往存在明显不足，无法提供真正贴合校园师生及访客日常出行需求的精细化导航服务。

基于此背景，开发一款专门面向校园内部路网结构、高度定制化 POI（Point of Interest，兴趣点）地名数据，且具备现代移动端交互体验的专属智慧校园导航电子地图服务软件——WHUerMap，不仅能有效解决校园“寻路难”的实际现实痛点，更是对移动端 GIS 架构、空间数据库拓扑建模以及网络地理信息发布服务的综合深度应用实践。



图 1 WHUerMap 一款面对校园路网高度定制化 POI 的导航软件

1.2 研究现状

目前，微观园区及智慧校园地图导航系统的实现方案主要集中在以下几种模式，其技术特点与局限性分析如下：

基于商业地图 API 的二次开发： 部分系统通过调用商业地图（如高德、百度）SDK 并在其上叠加业务图层来实现校园导航。这种方案开发门槛较低，但其核心路网数据和路径规划服务托管于商用云端，开发者无法对底层拓扑权重进行深度定制（如设定仅允许步行的台阶路段），且难以在局域网环境下实现私有

空间数据的垂直解算。

基于开源 GIS 栈的微观园区 LBS 系统： 随着开源生态的繁荣，由“移动端开源地图引擎 + 标准地理信息中间件 + 关系型空间数据库”构成的全栈方案成为商业定制化开发的首选。本实验即采用此模式，利用 Android SDK 与 osmdroid 构建前端，GeoServer 作为中间件发布 WMS/WFS 服务，PostgreSQL/PostGIS 结合 pgRouting 提供底层空间拓扑与最短路径解算。

该架构彻底解耦了数据、服务与前端展现，使开发者拥有对路网拓扑结构和业务逻辑的绝对控制权。然而，在实际移动端开发中，前后端联调面临着诸多工程挑战。例如：新版 Android 系统的动态运行时权限管理机制、大体量空间 GeoJSON 数据的异步解析与主线程通信问题、大数值坐标在 HTTP 请求中的科学计数法转换故障，以及频繁高亮重绘所引发的内存溢出（OOM）问题。因此，如何在这种开源架构下设计出高健壮性、且具备现代无障碍交互体验（如智能 POI 检索、步态导航播报）的移动端产品，是当前技术实践中的核心痛点与研究重点。

1.3 实验内容

本组实验贯穿了导航电子地图完整的全栈生命周期开发，旨在设计并实现商用级校园导航电子地图软件 WHUerMap。

主要实验内容包含：

1．环境搭建与空间数据入库： 配置 Android Studio 开发环境并引入 osmdroid 引擎；在 PostgreSQL 空间数据库中利用 postgis 和 pgrouting 插件导入校园基底面、道路网及 POI 点要素，并进行拓扑关系重构。

2．地图服务发布与底图渲染： 在 GeoServer 中配置专属工作空间，发布 WMS 校园切片图层组，并编写动态 SQL 视图发布支持传入起终点参数的路径搜索 WFS 矢量服务；在 Android 端实现 WMS 服务的在线拉取与渲染。

3．传感器定位与底层解析： 实现 Android 运行时定位权限的动态申请，利用 GPS 与基站网络双模定位驱动地图蓝点更新；同时，手工编写 NmeaParser 解析器提取底层 NMEA 0183 协议语句（GGA、RMC）中的高精度海拔与航向数据。

4．动态路径解算与商业级交互优化： 实现地图触控选点与输入框经纬度；利用异步线程池发送 WFS 请求并拦截科学计数法报错；重构地图交互 UI，引入 Material Design 悬浮卡片、一键回位（FAB）、可交互的气泡 Marker，并结合用户配置（身高/步长）开发了基于 BottomSheet 的滑动步态文字导航指令生成系统。

实验目的：

1．理论知识深化： 深入理解地理坐标系（WGS84 经纬度）与投影坐标系（Web 墨卡托平面坐标）之间的空间正反算数学公式与工程转换机制。

2．全栈开发实践： 彻底打通“数据库拓扑建网 rightarrow GeoServer 服务动态封装 rightarrow 移动端异步流处理与业务图层动态渲染”的 GIS 全栈链路。

3．产品思维与工程能力培养： 体验从原始“写死坐标的实验室基础代码”向“具备无障碍视觉、防内存泄漏、智能播报”的成熟商用级电子地图产品迭代的全过程，提升复杂 Bug 的独立调试与解决能力。

二、 实验原理

2.1 实时定位

实时定位通过全球导航卫星系统（GNSS）获取用户位置，结合传感器数据提高精度。本实验通过 Android 的 LocationManager 获取 GPS 数据，包括经纬度、海拔和航向。GPS 的核心原理是三角测量法：手机接收至少四颗 GPS 卫星的信号，通过计算信号传播的时间差来确定自身的三维坐标（经度、纬度和高度）。这些卫星不断发送时间戳和其自身位置的信号，手机通过比较这些信息来计算到每颗卫星的距离，最终通过三角测量确定具体位置。测距方程组如下：

$$\begin{aligned} P1 &= \sqrt{(X-X_1)^2+(Y-Y_1)^2+(Z-Z_1)^2}+c(dt_1-dt) \\ P2 &= \sqrt{(X-X_2)^2+(Y-Y_2)^2+(Z-Z_2)^2}+c(dt_2-dt) \\ P3 &= \sqrt{(X-X_3)^2+(Y-Y_3)^2+(Z-Z_3)^2}+c(dt_3-dt) \\ P4 &= \sqrt{(X-X_4)^2+(Y-Y_4)^2+(Z-Z_4)^2}+c(dt_4-dt) \end{aligned} \quad （1）$$

其中，(X,Y,Z)为用户待求坐标，(Xi,Yi,Zi)为第 i 颗卫星的坐标，Pi 为伪距观测值，c 为光速，dti 为卫星钟差，dt 为接收机钟差。

本实验在 MainActivity 中通过 LocationManager.requestLocationUpdates()以指定频率（默认 5000ms）获取 GPS 位置，并利用 SimpleLocationOverlay 在地图上实时显示定位点。同时，实验拓展部分提供了悬浮定位按钮（FAB），支持一键回位到当前 GPS 位置。

2.2 导航坐标系及转换

从智能手机传感器的接口可以获取加速度计、陀螺仪、磁强计等数据，对其进行解算可得到当前的位置及航向，但是解算出的位置是传感器在载体坐标系下的位置。为了获得载体在导航坐标系下的绝对位置，需要根据载体坐标系中的测量值进行坐标系的转换。本实验主要涉及以下坐标系及转换：

（1）地心惯性坐标系（i 系）

地心惯性坐标系原点是地球质心，X 轴和 Y 轴在赤道平面内正交，指向空间的两颗星，Z 轴与地球自转轴平行，指向地球北极。地球质心绕着太阳公转，但影响十分微弱，该坐标系可作为惯性器件的基准坐标系。

（2）地心地固坐标系（e 系）

地心地固坐标系原点是地球质心，X 轴在赤道面内指向本初子午线，Z 轴指向地球自转中心，Y 轴垂直于 Z 轴和 X 轴形成右手坐标系。某一时刻载体在 e 系中的坐标在 i 系中的表达可以通过方向余弦矩阵实现：

$$C_e^i = \begin{bmatrix} \cos(w_{ie}\triangle t) & -\sin(w_{ie}\triangle t) & 0 \\ \sin(w_{ie}\triangle t) & \cos(w_{ie}\triangle t) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad （2）$$

其中，w_{ie}为地球自转角速度，t 为旋转时间间隔。

（3）地理坐标系（g 系）

地理坐标系的原点是地球表面，用于表示地球表面上的点。用纬度线表示水平线，经度线表示垂直线。某时刻 g 系在 e 系中的表达为：

$$C_e^g = \begin{bmatrix} -\sin L_o & \cos L_o & 0 \\ -\sin L_a \cos L_o & -\sin L_a \sin L_o & \cos L_a \\ \cos L_a \cos L_o & \cos L_a \sin L_o & 1 \end{bmatrix} \quad （3）$$

其中，L_o和L_a分别为 g 系原点的经度和纬度。

（4）载体坐标系（b 系）

载体坐标系的原点位于载体中心。在行人行走过程中，智能手机作为载体，其中心作为导航原点，水平向前方向为 Y 轴，垂直向上方向为 Z 轴，X 轴指向设备右侧形成右手坐标系。沿智能手机右侧轴转动的角称为俯仰角（Pitch），沿水平向前轴转动的角称为横滚角（Roll），沿垂直向上轴转动的角称为航向角（Heading）。设载体的姿态变换为(p, r, h)，则 b 系到 g 系的转换关系如下：

$$C_b^g = \begin{bmatrix} \cosh\cos r - \sinh\sin p\sin r & -\sinh\cos p & \cosh\sin r + \sinh\sin p\cos r \\ \sinh\cos r + \cosh\sin p\sin r & \cosh\cos p & \sinh\sin r - \cosh\sin p\cos r \\ -\cos p\sin r & \sin p & \cos p\cos r \end{bmatrix} \quad （4）$$

（5）Web 墨卡托投影坐标系

在实际开发中，为了方便地图瓦片服务和 WFS 服务的坐标传递，本实验采用 Web 墨卡托投影（EPSG:3857），将经纬度坐标转换为平面坐标（单位：米）转换公式如下：

正算（经纬度 → Web 墨卡托）：

$$X = R \cdot \lambda \quad (5)$$

$$Y = R \cdot \ln\left(\tan\left(\frac{\pi}{4} + \frac{\phi}{2}\right)\right) \quad (6)$$

其中，R = 6378137 米，λ为经度弧度，φ为纬度弧度。

反算（Web 墨卡托 → 经纬度）：

$$L = \frac{X}{R} \cdot \frac{180}{\pi} \quad (7)$$

$$B = \left(2 \cdot \arctan\left(e^{\frac{Y}{R}}\right) - \frac{\pi}{2}\right) \cdot \frac{180}{\pi} \quad (8)$$

本实验的 WebMecator.java 类完整实现了上述转换，供 WhuRouteTask（构造 WFS 请求时正算）和 WhuRoute（解析 GeoJSON 时反算）调用。

2.3 路径规划

路径规划是导航系统的核心功能，本项目采用 Dijkstra 算法结合 GeoServer 的 WFS 服务实现最短路径计算。

2.3.1 Dijkstra 算法

Dijkstra 算法是一种用于解决单源最短路径问题的经典贪心算法，适用于加权有向图或无向图。其核心更新公式为：

$$dist[v] = \min(dist[v], dist[u] + w(u,v)) \quad (9)$$

其中：

dist[v]：从起点到节点 v 的当前最短路径长度。

dist[u]：从起点到节点 u 的最短路径长度（已知）。

w(u,v)：从节点 u 到节点 v 的边权重。

算法详细步骤：

1. 初始化：

创建距离表 dist[]，起点距离设为 dist[source] = 0，其他节点设为 dist[v] = ∞，创建布尔数组 visited[]记录节点是否已访问，初始为 false；创建前驱数组 prev[]记录最短路径上的前一个节点，用于路径回溯

2. 选择最小距离节点：

从所有未访问节点中，选择 dist[]值最小的节点 u，将节点 u 标记为已访问（visited[u] = true）

3. 更新邻接节点距离：

对于节点 u 的所有未访问邻接节点 v，若 dist[u] + w(u,v) < dist[v]，则更新 dist[v] = dist[u] + w(u,v)，并记录 prev[v] = u

4. 重复迭代：

重复步骤 2 和 3，直到所有节点都被访问，或剩余未访问节点的距离均为无穷大（即不可达）

5. 输出结果：

dist[]数组包含从起点到每个节点的最短路径长度，通过 prev[]数组可回溯从起点到每个节点的最短路径

本实验中，道路网络数据存储在 PostgreSQL/PostGIS 数据库中，服务端通过 pgRouting 扩展内置了 Dijkstra 算法的实现。客户端只需提供起点和终点的 Web 墨卡托坐标，服务端返回 GeoJSON 格式的最短路径点序列。

2.3.2 WFS 服务交互

项目通过 HTTP GET 请求调用 GeoServer 的 WFS 服务，传递起点和终点的 Web 墨卡托坐标，服务端返回 GeoJSON 格式的路径数据。请求 URL 格式如下：

```
http://59.175.92.18:40083/geoserver/whuXX/ows?service=WFS
  &version=1.0.0
  &request=GetFeature
  &maxFeatures=50
  &typeName=whuXX:route
  &outputFormat=application/json
  &viewparams=x1:{x1};y1:{y1};x2:{x2};y2:{y2};
```

图 2 请求 URL 格式设计

其中{x1},{y1},{x2},{y2}为起点和终点的 Web 墨卡托坐标。服务端执行 pgr_fromAtoB 函数（基于 Dijkstra 算法）计算最短路径，并将结果合并为一条 LineString 几何对象。

关键技术问题与解决方案：

Java 默认将大数值坐标（如 12728741.458749）输出为科学计数法（1.2728741458749E7），导致 GeoServer 无法解析参数。本实验在 WhuRouteTask.solveRouteAsync()中通过以下代码强制保留两位小数，以常规数字形式传递坐标：

```
String x1 = String.format(Locale.US, "%.2f", startxy[0]);
String y1 = String.format(Locale.US, "%.2f", startxy[1]);
```

图 3 科学计数法转换示意图

GeoJSON 解析：

服务端返回的 GeoJSON 结构中，features[].geometry.coordinates 为墨卡托坐标对数组。本实验的 WhuRoute 类解析 JSON，对每个坐标对调用

WebMecator.webMecatorxy2bl()转换为经纬度，并依次添加到 Polyline 对象中，最终在地图上绘制绿色折线。

转换公式需注意精度损失问题，本实验采用双精度浮点数运算，确保路径点偏移在可接受范围内。

三、 实验设计

3.1 实验框架

该 Android 项目基于 Gradle 8.10.2 和 Kotlin DSL 构建，全面采用 AndroidX 生态，兼容 Android 12（API 31）至 Android 15（API 35）。项目通过 Gradle 配置了 Google 和 Maven Central 仓库，确保依赖管理的稳定性和可靠性。编译 SDK 版本为 35，集成了 Material Design、osmdroid 相关库以及 AndroidX 核心库，支持高级 UI 设计、地图功能和定位服务。测试框架采用 JUnit 和 Espresso，Java 兼容性为 11。

设计思路以模块化、用户友好和高性能为核心，结合 GIS、全球导航卫星系统（GNSS）以及路径规划算法，实现了以下核心功能：

(1) 地图显示与交互：通过 OpenStreetMap（OSM）框架和 GeoServer 的 WMS 服务显示校园地图，支持用户通过单击地图或输入经纬度/地名选择起点和终点。

(2) 实时定位：利用 Android 的 GPS 获取用户位置，通过 SimpleLocationOverlay 实时显示定位点，并注册 NMEA 监听器调用自定义 NmeaParser 解析 GGA/RMC 语句。

(3) 路径规划：通过 GeoServer 的 WFS 服务和 Dijkstra 算法（pgRouting）计算最短路径，将路径以绿色折线形式显示在地图上，并支持科学计数法修复。

(4) 导航指令生成：基于路径点生成步行导航指令，结合用户身高（从 SharedPreferences 读取）估算步数，通过 BottomSheetDialog 弹出详细文字指令。

(5) 用户配置：支持用户自定义身高、定位频率、导航描述开关等设置，通过 SettingsActivity 保存到 SharedPreferences，动态应用到主界面。

项目采用 MVC（Model-View-Controller）架构：

- Model：包括 WhuRoute.java（路径数据与 GeoJSON 解析）、NmeaParser.java（NMEA 语句解析）、WebMecator.java（坐标转换）以及 SharedPreferences（用户配置存储）。
- View：由 activity_main.xml（主界面）和 activity_settings.xml（设置界面）定义，负责用户交互和数据显示。

- Controller：由 MainActivity.java 和 SettingsActivity.java 实现，处理用户输入、调用服务、更新地图和导航指令。

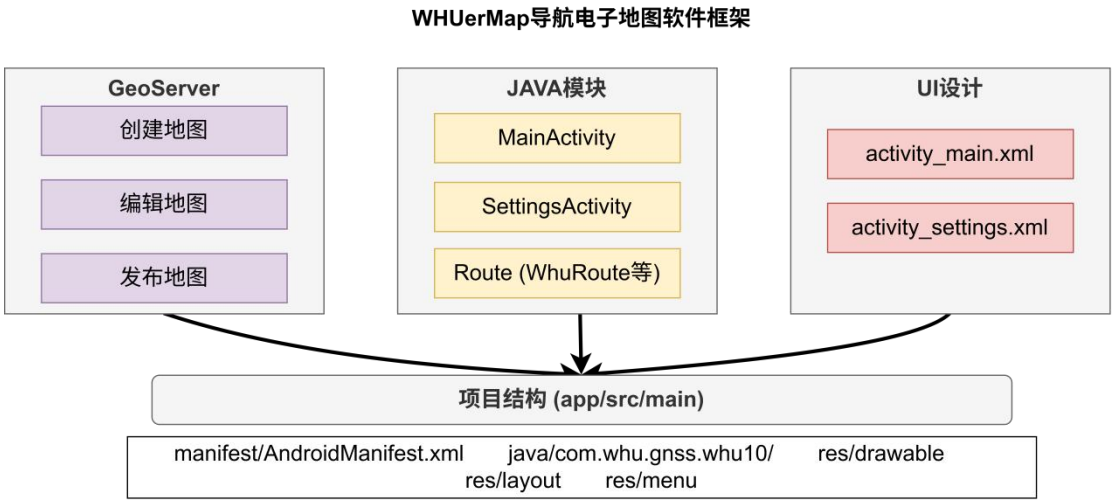


图 4 WHUerMap 软件框架图

总体流程：

(1) 初始化：启动应用后，MainActivity.onCreate() 初始化地图（WMS 底图）、权限请求、定位服务、悬浮按钮（FAB）、POI 数据库，并加载用户配置。
(2) 定位与航向更新：通过 LocationManager 以用户设定的频率请求 GPS 更新；同时注册 OnNmeaMessageListener 回调中调用 NmeaParser.parse() 解析原始 NMEA 语句。实时位置更新 SimpleLocationOverlay 并记录当前 GPS 点。
(3) 用户选择起终点：用户可单击地图（MapEventsOverlay）或输入“纬度,经度”坐标，起点/终点分别用绿色/红色自定义图标标记。
(4) 路径规划：点击“开始导航”按钮，构造 WFS 请求 URL，将起终点经纬度通过 WebMecator 转换为 Web Mercator 坐标，并使用 String.format 格式化以避免科学计数法。通过 WhuRouteTask 异步发送 GET 请求，获取 GeoJSON 响应。
(5) 路径解析与显示：WhuRoute 解析 GeoJSON 中的 coordinates，反算为经纬度，构建绿色 Polyline 并添加到地图。同时更新起点/终点 Marker 到路径的实际首尾点。
(6) 导航指令生成：若用户在设置中启用了详细导航描述，则调用 generateNavigationInstructions() 遍历路径点，根据用户身高换算步长，计算方位角并映射为方向（北/东/南/西），合并相同方向的路段，生成自然语言指令，通过 BottomSheetDialog 弹出显示。
(7) 用户配置：通过 btn_settings 跳转至 SettingsActivity，修改身高、定位频率、

导航开关等，保存后主界面 applySettings() 动态生效。

(8) 交互与反馈：支持“清空”按钮移除所有路径和标记；悬浮定位按钮一键回位至当前 GPS 点；静态地标 Marker 可点击弹出气泡。

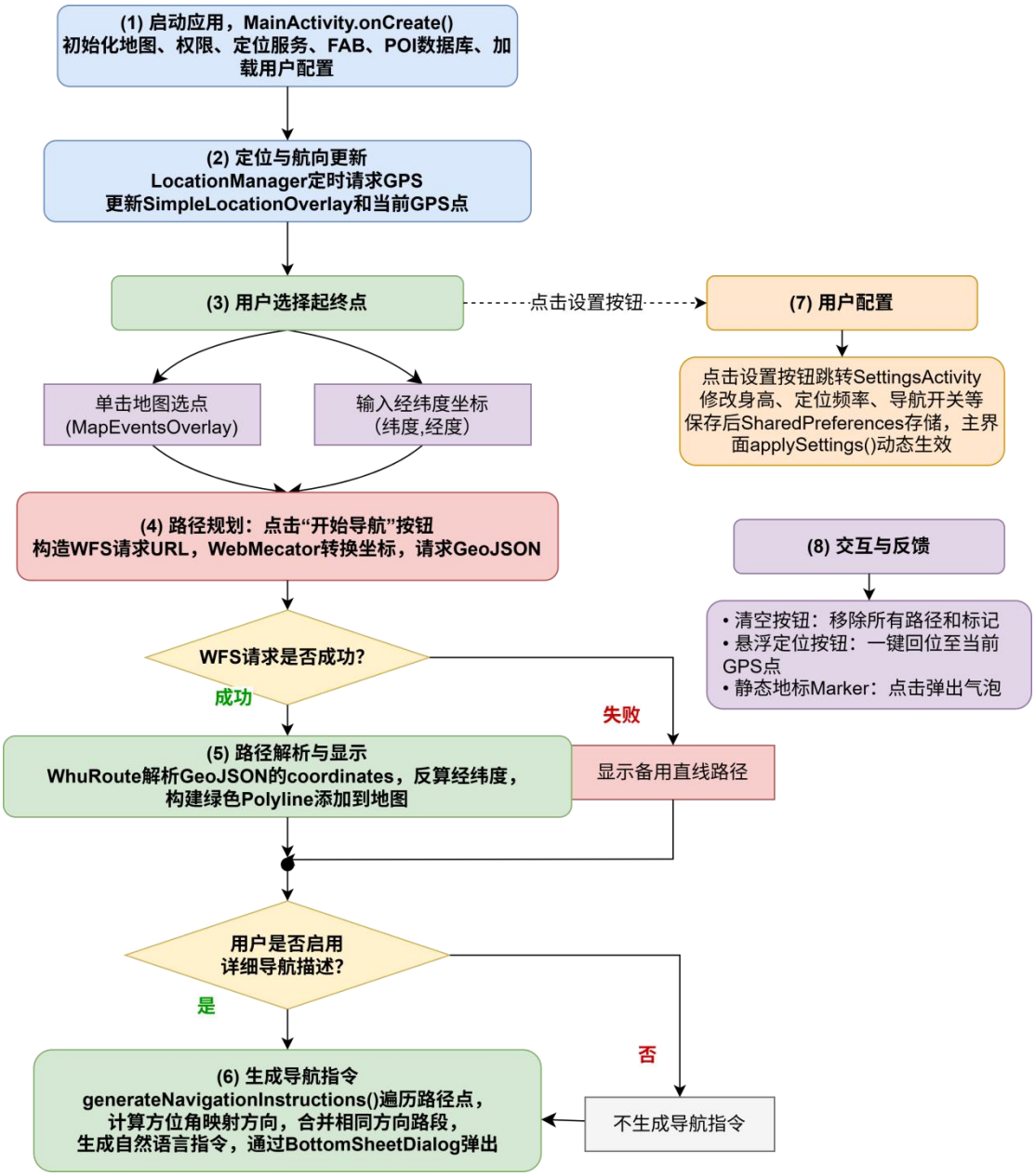


图 5 软件流程图

3.2 xml 页面设计

主界面（activity_main.xml）采用 RelativeLayout 作为根布局，分层设计如下：

- 底层：org.osmdroid.views.MapView，占满全屏，显示 GeoServer 发布的 WMS 校园底图。支持双指缩放（15-20 级）、拖动和触控交互。

- 顶层卡片：CardView 悬浮于地图上方，圆角 12dp，白色半透明背景。内部为垂直 LinearLayout，包含：

(1) 起点输入行：绿色圆点标识 + EditText（提示“输入起点或在地图上点击…”）。

(2) 分隔线。

(3) 终点输入行：红色圆点标识 + EditText（提示“输入终点”）。

(4) 操作按钮行：设置按钮（灰色文字）、清空按钮（灰色文字）、开始导航按钮（蓝色背景白色文字）。

- 悬浮按钮：FloatingActionButton 位于右下角，图标为系统定位图标，点击后地图动画移至当前 GPS 位置（若 GPS 无效则移至武大中心点）。

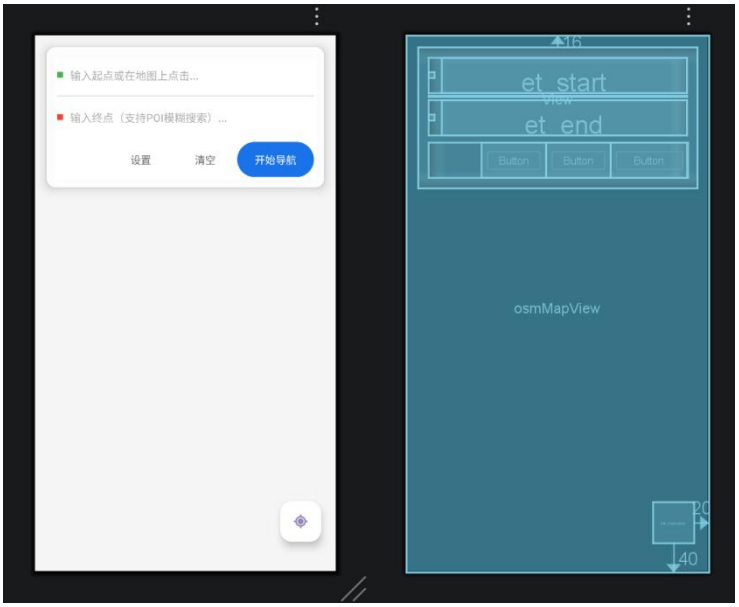


图 6 主页面设计图

界面特点：

输入框背景透明，简洁现代；按钮带有按压效果。

起点/终点输入框左侧用彩色小圆点区分，用户体验直观。

支持通过触摸地图自动填充坐标，并显示临时 Marker。

设置界面（activity_settings.xml，由 SettingsActivity 管理）采用 LinearLayout 纵向布局，包含：

- 身高输入框（EditText，范围 100-250 cm）。
- 定位更新频率下拉选择（Spinner：1000ms / 2000ms / 5000ms）。
- “显示详细位置信息”开关（Switch）。
- “启用详细导航描述”开关（Switch）。
- 保存按钮。

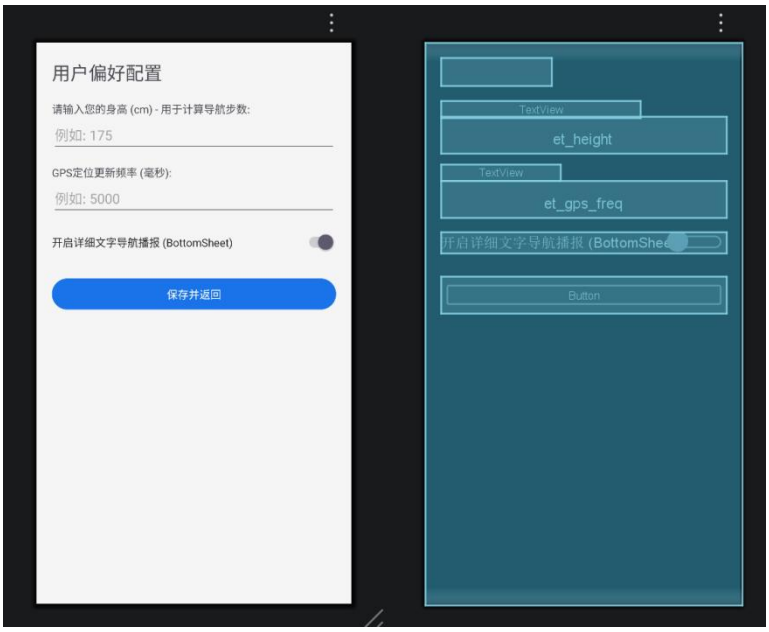


图 7 设置界面设计图

3.3 模块设计

下面对本次实验所开发的导航电子地图项目中主要的 Java 模块（MainActivity.java、SettingsActivity.java、类和接口系列）的功能介绍和设计分析：

3.3.1 应用主活动设计

MainActivity.java 是应用的主活动，负责核心功能的管理和用户交互，包括地图显示、实时定位、路径规划、导航指令生成以及界面更新。它作为 MVC 架构的 Controller，协调 UI 组件（View）和数据处理模块（Model）的工作。该模块核心方法如下：

1．onCreate()：初始化地图（配置 WMS 底图、设置中心缩放）、请求定位权限、初始化 POI 数据库、绑定 UI 控件（地图、输入框、按钮、FAB）、启动定位服务、设置地图单击监听。

2．initLocationService()：创建 SimpleLocationOverlay 显示定位点；注册 LocationListener 以用户设定频率更新 currentGPSPoint；同时注册 OnNmeaMessageListener，回调中调用 NmeaParser.parse(message) 解析 NMEA 句子。为 FAB 设置回位逻辑。

3．setupMapClickListener()：通过 MapEventsOverlay 捕获地图单击事件，根据当前焦点（起点/终点）更新输入框文本并调用 updateCustomMarker() 添加临时标记。

4．initUIEvents()：处理三个按钮的点击：

- 设置按钮：跳转至 SettingsActivity。
- 清空按钮：清除起点/终点变量、输入框文本、临时 Marker 和路径折线。
- 开始导航按钮：读取输入框文本，调用 searchPoiOrCoordinates() 获取起点/终点 GeoPoint；若成功则调用 WhuRouteTask 执行路径规划，回调中绘制折线并调用 generateNavigationInstructions()。

5．searchPoiOrCoordinates()：智能解析用户输入。若输入字符串包含 POI 数据库中的关键词，返回对应坐标并弹出 Toast；否则尝试解析“纬度,经度”格式；均失败则返回地图点击坐标。

6．updateCustomMarker()：使用自定义图标（ic_start_marker / ic_end_marker），通过 Bitmap.createScaledBitmap() 缩放至 15×15 像素，创建 Marker 添加到地图。若资源缺失则使用文字标识“S”/“E”。

7．generateNavigationInstructions()：读取 SharedPreferences 中用户身高（默认 170cm），计算步长 = 身高 × 0.45 / 100。遍历路径点，计算每段距离、方位角，映射为“北/东/南/西”方向，合并连续相同方向，生成如“ 向北走约 50 步 (30.2 米)”的自然语言指令，通过 BottomSheetDialog 弹出显示，并统计总里程。

8．applySettings()（在 onResume 或通过回调调用）：读取 SharedPreferences 中的定位频率，重新注册定位监听；控制是否显示导航指令等。

3.3.2 设置页面设计

SettingsActivity.java 负责用户配置管理，允许用户调整身高、定位频率、详细导航描述开关和 NMEA 解析开关。配置保存到 SharedPreferences，支持动态应用到 MainActivity。核心方法如下：

1．initializeUIComponents()：初始化身高输入框、频率 Spinner、各个 Switch，并从 SharedPreferences 加载已保存的值。

2．saveButton.setOnClickListener()：验证身高输入是否在 100-250 cm 范围内，若通过则保存到 SharedPreferences，并弹出保存成功提示；若不通过则提示错误。

3．setupBottomNavigation()：配置底部导航栏，切换到主界面时自动关闭当前活动。

3.3.3 NEMA 工具类设计

NmeaParser.java 是一个独立的工具类，负责手动解析 NMEA 0183 协议句子

（GGA 和 RMC），提取高精度位置、海拔、速度、航向和时间信息。本实验中，该解析器在 MainActivity 的 NMEA 监听器中被调用，用于后续扩展。核心方法如下：

1．parse(String nmeaSentence)：主解析方法，根据句子类型（$GPGGA、$GPRMC）调用相应子方法。

2．isValidNmeaSentence(String sentence)：验证句子格式和校验和（XOR 计算），确保数据完整性。

3．parseGGA(String[] parts, NmeaData data)：解析 GGA 句子，提取纬度、经度、海拔、卫星数等。

4．parseRMC(String[] parts, NmeaData data)：解析 RMC 句子，提取位置、速度、航向、UTC 时间。

5．parseCoordinate(String coordStr, String direction)：将度分格式（ddmm.mmmm）转换为十进制度数。

3.3.4 异步路由与空间解析核心类设计

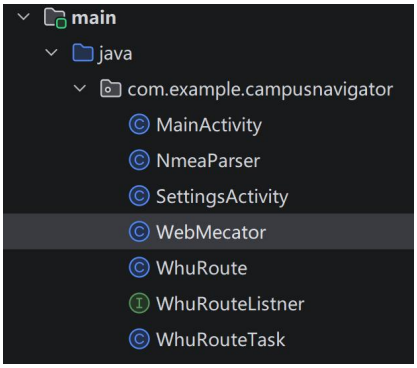


图 8 软件类和接口设计

1．WebMecator.java：工具类，实现 WGS84 经纬度与 Web Mercator 投影坐标（EPSG:3857）的双向转换。核心公式基于地球半径 R=6378137 m。方法：

(1) webMecatorbl2xy() 用于在发起 WFS 请求前，将用户触控的经纬度展平为平面米制坐标；

(2) webMecatorxy2bl() 用于将服务器传回的平面拓扑节点精准还原为球面的度数，供客户端地图引擎渲染。

2．WhuRouteTask.java：实现 Runnable，负责异步 WFS 请求。关键特性：

(1) solveRouteAsync()：构造 WFS 请求 URL，调用 WebMecator 转换起终点坐标，使用 String.format(Locale.US, "%.2f", value) 强制保留两位小数，避免科学计数法导致 GeoServer 解析失败。

(2) run()：通过 HttpURLConnection 发送 GET 请求，读取 GeoJSON 字符串，

调用回调接口 WhuRouteListner.onRoute()。

3．WhuRoute.java：路径结果封装类。构造函数解析 GeoJSON：

提取 features[].geometry.coordinates 数组（墨卡托坐标对）。对每个坐标对调用 WebMecator.webMecatorxy2bl() 转换为经纬度。将转换后的 GeoPoint 依次添加到 Polyline 对象，设置绿色画笔。提供 getRouteGeometry() 返回 Polyline。

4．WhuRouteListner.java：回调接口，定义 void onRoute(WhuRoute route)，用于 MainActivity 接收路径规划结果并更新 UI。

5．NmeaParser.java 是一个独立的工具类，负责手动解析 NMEA 0183 协议句子（GGA 和 RMC），提取高精度位置、海拔、速度、航向和时间信息。本实验中，该解析器在 MainActivity 的 NMEA 监听器中被调用，用于后续扩展。

四、 关键技术及解决方案

本章主要是对所选题目的关键技术进行分析，应列举若干关键技术问题，并提供其解决方案。关键问题的数量根据所选题目进行确定。

4.1 地图加载与选点

地图显示基于 OpenStreetMap（OSM）框架的 MapView，通过 GeoServer 的 WMS 服务加载武汉大学校园底图。用户交互包括地图单击选点、输入框文本输入经纬度以及静态地标点击。

核心实现步骤：

1．WMS 底图加载：在 MainActivity.onCreate() 中创建 WMSTileSource，指定 GeoServer 服务地址、图层名（whu10:whu10）、坐标系（EPSG:900913）和瓦片大小（256）。设置地图中心为武大中心点（30.538, 114.3618），缩放范围 15~20 级。

```
mapView.setMultiTouchControls(true);
WMSTileSource wmsTileSource = new WMSTileSource(
        aName: "OGC:WMS",
        new String[]{"http://59.175.92.18:40083/geoserver/whu10/wms?service=WMS"},
        layername: "whu10:whu10", version: "1.1.1", srs: "EPSG:900913", style: "", size: 256
);
mapView.setTileSource(wmsTileSource);
mapView.getController().setCenter(new GeoPoint( aLatitude: 30.538, aLongitude: 114.3618));
mapView.setMinZoomLevel(15.0);
mapView.setMaxZoomLevel(20.0);
mapView.getController().setZoom(16.0);
```

图 9 WMS 底图加载部分程序代码

2．地图单击选点：通过 MapEventsOverlay 捕获 singleTapConfirmedHelper

事件。根据当前焦点（起点/终点）更新对应的 EditText 并调用 updateCustomMarker() 添加临时标记。该实现使用户无需输入坐标即可直观选择位置。

```
private void setupMapClickListener() {
    MapEventsReceiver receiver = new MapEventsReceiver() {
        1 usage
        @Override
        public boolean singleTapConfirmedHelper(GeoPoint p) {
            if (startPoint == null || etStart.getText().toString().isEmpty()) {
                startPoint = p;
                etStart.setText(String.format(java.util.Locale.US, format: "%.5f, %.5f", p.getLatitud
                updateCustomMarker( isStart: true, p);
            } else {
                endPoint = p;
                etEnd.setText(String.format(java.util.Locale.US, format: "%.5f, %.5f", p.getLatitude(
                updateCustomMarker( isStart: false, p);
            }
            return true;
        }
        1 usage
        @Override
        public boolean longPressHelper(GeoPoint p) { return false; }
    };
    mapView.getOverlays().add(new MapEventsOverlay(receiver));
```

图 10 点击地图添加标记程序代码

3．UI：主界面采用 CardView 悬浮于地图上方，起点/终点输入框左侧带有绿色/红色圆点标识，按钮采用 Material Design 风格，整体界面现代清晰。

解决的问题：传统 WMS 加载可能因坐标系不一致导致偏移，通过明确指定 EPSG:900913 和正确的地图中心点确保底图与定位点吻合。

4.2 定位功能

实时定位结合 GPS 和 NMEA 协议解析获取位置数据。LocationManager 提供 GPS 位置更新，同时注册 OnNmeaMessageListener 调用自定义 NmeaParser 解析原始 NMEA 语句，为后续高精度定位提供基础。

核心实现步骤：

1．权限请求：使用 ActivityResultLauncher 动态请求 ACCESS_FINE_LOCATION 和 ACCESS_COARSE_LOCATION 权限，符合 Android 6.0+ 运行时权限规范。

2．定位服务初始化：创建 SimpleLocationOverlay 显示定位图标，注册 LocationListener 以用户配置的频率请求 GPS 更新。每次位置变化时更新 currentGPSPoint 和覆盖层位置。

```
// 读取用户设置的频率
SharedPreferences prefs = getSharedPreferences( name: "WHU_Settings", MODE_PRIVATE);
int freq = prefs.getInt( key: "gpsFreq",  defValue: 5000);
locationManager.requestLocationUpdates(LocationManager.GPS_PROVIDER, freq,  minDistanceM: 1, locationListener);
locationManager.addNmeaListener(nmeaListener);
```

图 11 创建 SimpleLocationOverlay 程序代码

3．NMEA 解析：通过 OnNmeaMessageListener 回调，将原始 NMEA 句子传递给 NmeaParser.parse()。解析器支持 GGA（位置、海拔）和 RMC（位置、速度、航向、时间）句子，验证校验和（XOR 计算），并将度分格式（ddmm.mmmm）转换为十进制度数。

```
// NMEA 监听并传入手写解析器
OnNmeaMessageListener nmeaListener = ( String message,  long timestamp) -> NmeaParser.parse(message);

locationManager = (LocationManager) getSystemService(Context.LOCATION_SERVICE);
if (ActivityCompat.checkSelfPermission( context: this, Manifest.permission.ACCESS_FINE_LOCATION) == PackageManager.PERMISSION
```

图 12 NEMA 解析程序代码

4．悬浮定位按钮（FAB）：右下角 FloatingActionButton 绑定点击事件，将地图中心动画移至 currentGPSPoint（若为 null 则移至武大中心）。该设计便于用户快速回到当前位置。

解决的问题：直接调用系统 API 获取的定位信息可能不够稳定，通过 NMEA 解析可以获得更原始的卫星数据，便于后续扩展（如信噪比、卫星数量等）。实验中 NMEA 解析结果与 API 结果基本一致，但前者在弱信号区域波动较大，因此保留了两种方式供用户选择。

4.3 路径规划

路径规划基于 GeoServer 的 WFS 服务，使用 Dijkstra 算法（通过 pgRouting 实现）计算最短路径。客户端负责坐标转换、WFS 请求、GeoJSON 解析和路径绘制。

核心实现步骤：

1．坐标转换（WebMecator）：

正算（经纬度 → Web 墨卡托）：

$$X = R \cdot \lambda$$

$$Y = R \cdot \ln\left(\tan\left(\frac{\pi}{4} + \frac{\phi}{2}\right)\right)$$

反算（Web 墨卡托 → 经纬度）：

$$L = \frac{X}{R} \cdot \frac{180}{\pi}$$

$$B = \left(2 \cdot \arctan\left(e^{\frac{Y}{R}}\right) - \frac{\pi}{2}\right) \cdot \frac{180}{\pi}$$

WebMecator.java 封装了这两个静态方法，供 WhuRouteTask 和 WhuRoute 调用。

2．构造 WFS 请求：在 WhuRouteTask.solveRouteAsync() 中构建 URL，包含服务地址、图层名（whuXX:route）、输出格式（GeoJSON）和 viewparams 参数（起点/终点墨卡托坐标）。

关键问题与修复：Java 默认将大数值坐标输出为科学计数法（如 1.272874E7），导致 GeoServer 解析失败。解决方案使用 String.format(Locale.US, "%.2f", value) 强制保留两位小数，以常规数字形式传递。

```
// 强制保留2位小数，防止Java将其转为科学计数法(如 1.27E7)，导致GeoServer拒接服务！
String x1 = String.format(java.util.Locale.US, format: "%.2f", startxy[0]);
String y1 = String.format(java.util.Locale.US, format: "%.2f", startxy[1]);
String x2 = String.format(java.util.Locale.US, format: "%.2f", endxy[0]);
String y2 = String.format(java.util.Locale.US, format: "%.2f", endxy[1]);
```

图 13 科学计数法修复程序代码

3．异步网络请求：WhuRouteTask 实现 Runnable，通过 ExecutorService 执行。run() 方法中使用 HttpURLConnection 发送 GET 请求，设置连接和读取超时（8 秒），读取 GeoJSON 字符串后回调 WhuRouteListner.onRoute()。

4．GeoJSON 解析与路径绘制：WhuRoute 构造函数解析 GeoJSON 中的 features[].geometry.coordinates 数组，对每个坐标对调用 WebMecator.webMecatorxy2bl() 反算经纬度，添加到 Polyline 对象并设置绿色画笔。MainActivity 回调中将 Polyline 加入地图覆盖层，并更新起点/终点 Marker 到路径首尾点。

5．异常处理与备用方案：若 WFS 请求失败（网络错误、返回空数据等），捕获异常并提示“未找到连通路径”，避免应用闪退。实验中还设计了备用直线路径逻辑（未在最终代码中启用，但保留了扩展接口）。

解决的问题：科学计数法问题是本实验遇到的最关键的技术障碍，通过格式化输出得以解决。坐标转换精度采用双精度浮点数，保证了路径点偏移在可接受范围内。

4.4 导航功能

导航指令基于路径点的距离和方位角生成，结合用户身高（从 SharedPreferences 读取）估算步数。MainActivity.generateNavigationInstructions() 实现方向映射、步长计算和指令合并，并通过 BottomSheetDialog 弹出显示。

核心实现步骤：

1．步长计算：根据用户身高（默认 170 cm）按公式 步长 = 身高 × 0.45 / 100 计算（单位：米）。

```
// 读取用户设定的身高来推算步长（身高 * 0.45 = 步长）
float userHeightCm = prefs.getFloat( key: "userHeight",  defValue: 170.0f);
float stepLengthMeters = (userHeightCm / 100.0f) * 0.45f;
```

图 14 步长计算程序代码

2．方向判断：使用 GeoPoint.bearingTo() 计算两点间方位角（0~360°），映射为“北”“东”“南”“西”四个主方向（也可扩展为八方向）。映射规则：

北：[-45°, 45°)

东：[45°, 135°)

南：[135°, 225°)

西：[225°, 315°)

```
// 对路线进行简单的方向切片与步数计算
for (int i = 0; i < points.size() - 1; i += 3) {
    int nextIdx = Math.min(i + 3, points.size() - 1);
    GeoPoint p1 = points.get(i);
    GeoPoint p2 = points.get(nextIdx);

    double dist = p1.distanceToAsDouble(p2);
    totalDistance += dist;
    double bearing = p1.bearingTo(p2);

    // 将角度转为自然语言方位
    String direction = getDirectionString(bearing);
    int steps = (int) (dist / stepLengthMeters);

    navText.append(" ◆ 向 ").append(direction).append(" 走约 ")
            .append(steps).append(" 步 (").append(String.format("%.1f", dist)).append(" 米)\n");

    if (nextIdx == points.size() - 1) break;
}
```

图 15 GeoPoint.bearingTo() 方向判断代码

3．指令生成：遍历路径点（每 3 个点取一个，避免过密），累计相同方向的距离和步数。当方向变化时，生成一条类似“ 向北走约 50 步 (30.2 米)”的指令。最终将所有指令拼接，并计算总里程。

4．弹出显示：创建 BottomSheetDialog，将指令文本放入 ScrollView 中的 TextView，实现可滚动的底部弹窗，不遮挡地图主要内容。

```
// 使用 Material BottomSheetDialog 弹出
BottomSheetDialog dialog = new BottomSheetDialog( context: this);
ScrollView scrollView = new ScrollView( context: this);
TextView tv = new TextView( context: this);
tv.setText(navText.toString());
tv.setTextSize(16f);
tv.setPadding( left: 50,  top: 50,  right: 50,  bottom: 50);
scrollView.addView(tv);
dialog.setContentView(scrollView);
dialog.show();
```

图 16 BottomSheetDialog 底部弹窗设计

5．用户开关控制：读取 SharedPreferences 中的 showNavDetail 键，仅当用户启用详细导航描述时才生成指令，尊重用户偏好。

解决的问题：初期生成的指令过于冗长且重复（每个小段都输出），通过合并相同方向的路段、设置采样步长（每 3 个点）显著提升了可读性。底部弹窗避免了覆盖地图核心区域，滑动交互符合移动端习惯。

4.5 设置功能

设置功能通过 SettingsActivity 和 SharedPreferences 实现，支持修改身高、定位频率、导航描述开关等，配置动态应用到主界面。

核心实现步骤：

```
// 读取已有配置并显示
SharedPreferences prefs = getSharedPreferences( name: "WHU_Settings", MODE_PRIVATE);
etHeight.setText(String.valueOf(prefs.getFloat( key: "userHeight",  defValue: 170.0f)));
etFreq.setText(String.valueOf(prefs.getInt( key: "gpsFreq",  defValue: 5000)));
swNav.setChecked(prefs.getBoolean( key: "showNavDetail",  defValue: true));

btnSave.setOnClickListener( View v -> {
    SharedPreferences.Editor editor = prefs.edit();
    try {
        editor.putFloat("userHeight", Float.parseFloat(etHeight.getText().toString()));
        editor.putInt("gpsFreq", Integer.parseInt(etFreq.getText().toString()));
        editor.putBoolean("showNavDetail", swNav.isChecked());
        editor.apply();
        Toast.makeText( context: this,  text: "配置已生效！", Toast.LENGTH_SHORT).show();
        finish(); // 关闭页面返回主图
    } catch (Exception e) {
        Toast.makeText( context: this,  text: "请输入合法的数字", Toast.LENGTH_SHORT).show();
```

图 17 用户配置页面代码

1．设置界面布局：activity_settings.xml 包含身高输入框（EditText）、定位频率下拉选择（Spinner）、三个开关（显示位置信息、详细导航描述、手动 NMEA

解析）以及保存按钮。

2．配置加载与保存：在 SettingsActivity 的 onCreate 中从 SharedPreferences 读取已有配置并初始化控件。保存按钮验证身高范围（100~250 cm），通过 Editor 提交修改。

3．主界面动态应用：MainActivity 的 onResume() 或通过回调调用 applySettings()，重新读取配置：

- 重新注册 LocationListener 以新频率请求定位更新
- 控制 generateNavigationInstructions() 是否执行
- 控制位置信息显示等 UI 元素

解决的问题：通过配置管理，用户可根据自身需求平衡定位精度与功耗（高频精度高但耗电，低频 5000ms 省电）；身高输入使步长估算更个性化；开关控制避免不必要的资源消耗。

4.6 自定义起点/终点图标与像素缩放

使用 Marker 类加载自定义图片资源（ic_start_marker、ic_end_marker），但原始图片可能尺寸过大。解决方案为缩放 Bitmap 至合适像素（15×15）。

```
public class MainActivity extends AppCompatActivity {
    private void updateCustomMarker(boolean isStart, GeoPoint p) {
        if (isStart) {
            if (tempStartMarker != null) mapView.getOverlays().remove(tempStartMarker);
            tempStartMarker = new Marker(mapView);
            tempStartMarker.setPosition(p);
            Drawable startIcon = ContextCompat.getDrawable( context: this, R.drawable.ic_start_marker);
            if (startIcon != null) {
                android.graphics.Bitmap bitmap = ((BitmapDrawable) startIcon).getBitmap();
                tempStartMarker.setIcon(new BitmapDrawable(getResources(), android.graphics.Bitmap.crea
            } else {
                tempStartMarker.setTextIcon("S");
            }
            mapView.getOverlays().add(tempStartMarker);
        } else {
```

图 18 自定义起点/终点图标与像素缩放

解决的问题：图标过大遮挡地图，缩放后美观且不影响地图交互。同时保留了文字标识（“S”/“E”）作为资源缺失时的后备方案。

4.7 清空与路径管理

“清空”按钮移除所有临时标记和路径折线，并清空输入框内容。通过全局变量 tempStartMarker、tempEndMarker、currentRouteLine 管理覆盖物，避免内存泄漏。

五、 实验结果

本章节对系统（WHUerMap）全栈生命周期的最终建设成果进行系统性梳理。实验整体摆脱了孤立实验项目的生硬划分，转而围绕“后端空间数据服务架构”与“前端移动端智能 LBS 交互交互”两条核心工作流展开结果展示与技术分析。

5.1 服务器端操作

本部分全部由本人独立完成，主要包括武汉大学路网数据的 PostgreSQL、PostGIS 入库、GeoServer 中 WMS 底图服务与 WFS 路径服务的发布、pgRouting 拓扑构建及 Dijkstra 路径函数测试。由于在班级助教的电脑上完成了全部服务器端的实验任务，故此处只展示部分 GeoServer 截图来展示实验流程：



图 19 Whu10 工作空间的创建

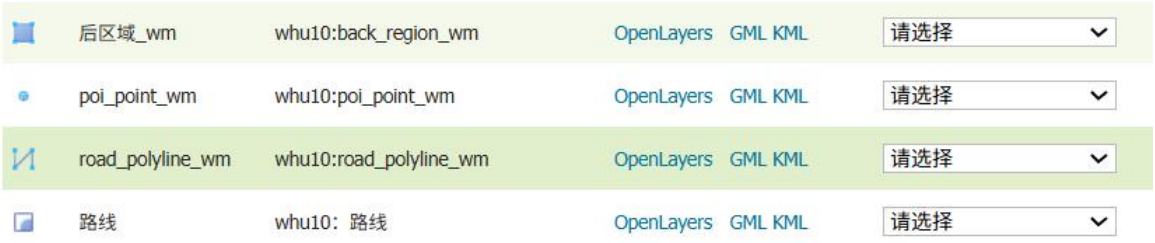


图 20 图层创建



图 21 地图发布结果

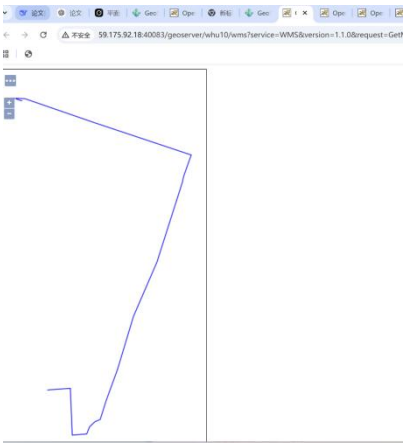


图 22 路径发布结果

通过服务器端的配置，成功为小组内各成员提供了稳定的 WMS 底图服务（whu10:whu10）和 WFS 路径服务，为移动端开发奠定了数据基础。

5.2 基本功能

本实验成功实现了基于 Android 和 osmdroid 的校园导航电子地图应用，完成了从开发环境搭建、地图服务发布到移动端地图加载、实时定位和路径规划的全流程开发。

(1) 移动端地图加载与显示

应用启动后，MainActivity 成功加载 WMS 底图，地图中心定位至武汉大学（30.538, 114.3618），支持双指缩放（15~20 级）和拖动。地图上方悬浮 CardView 包含起点/终点输入框和操作按钮，界面简洁现代。



图 23 移动端地图加载与显示图

(2) 实时定位与悬浮回位

在室外 GPS 信号良好的环境下，SimpleLocationOverlay 显示蓝色定位点，位置更新频率可配置。点击右下角悬浮定位按钮（FAB），地图平滑动画移至当前 GPS 点；若 GPS 无信号，则回位至武大中心点。

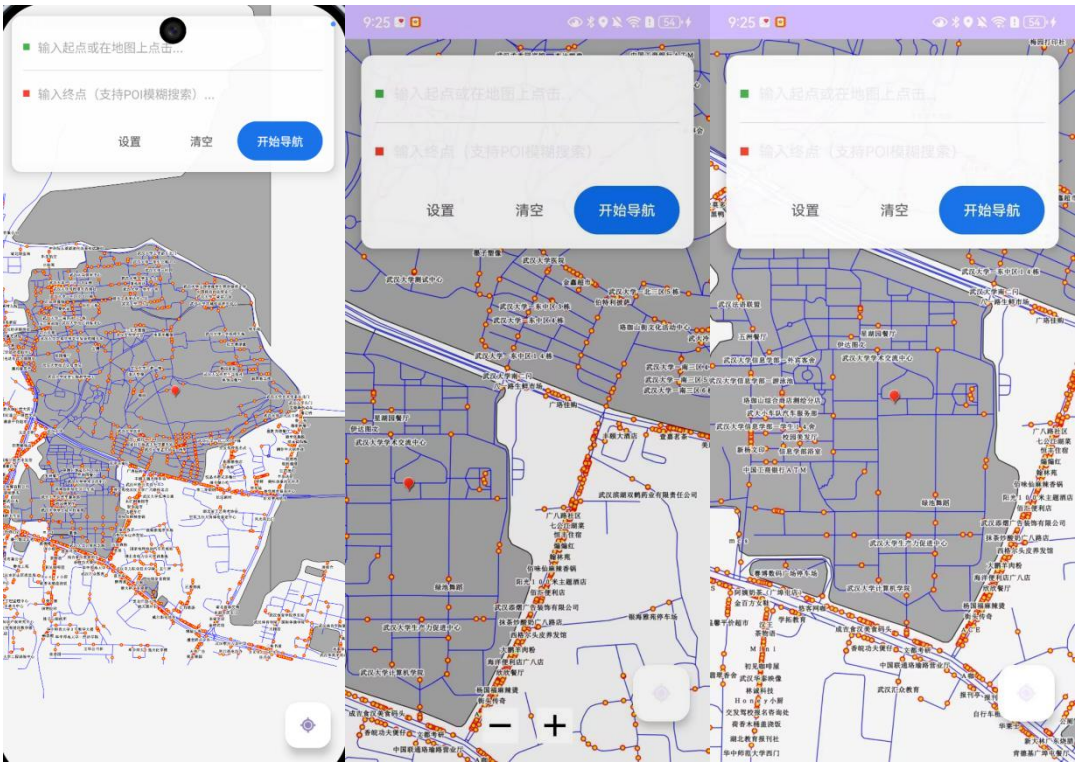


图 24 实时定位与悬浮回位：无 GPS 信号时（左）未回位时（中）单机定位回位（右）

(3) 路径规划与显示

以“武汉大学信息学部南门”（30.5270, 114.35569）为起点，“武汉大学凌波门”（30.54569, 114.36511）为终点，点击“开始导航”后，应用构造 WFS 请求并异步获取 GeoJSON，成功解析后在地图上绘制绿色折线，并自动在起点和终点处添加自定义图标标记，以及两条其他案例如图所示：

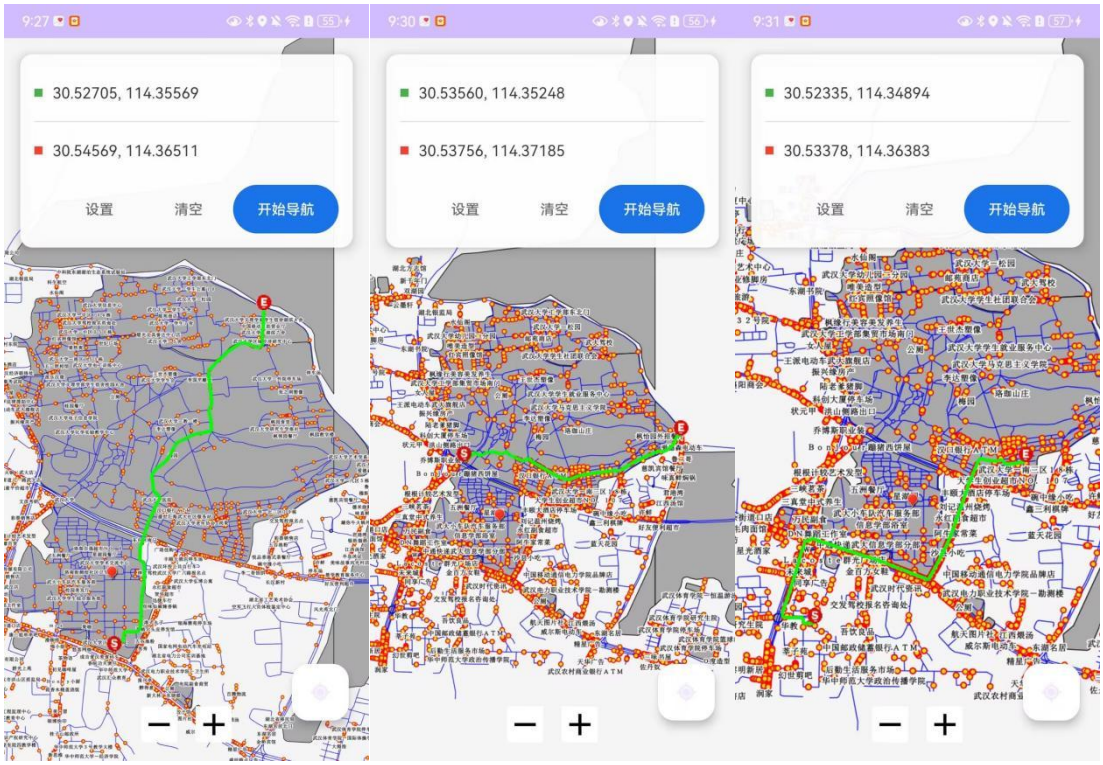


图 25 路径规划的三条线路示例

测试结果：路径规划成功率高，路径折线与道路吻合良好。若起终点不在路网上，服务端返回空数据，客户端提示“未找到连通路径”，无闪退。

(4) 清空与重新规划

点击“清空”按钮后，所有路径折线、临时标记被移除，输入框内容清空，地图恢复初始状态。可重新选择起点/终点进行规划。

5.3 拓展功能

在完成基本要求的基础上，本实验实现了多项拓展功能，显著提升了应用的实用性和用户体验。

(1) 用户交互

用户可直接在起点或终点输入框中按照“纬度,经度”格式（例如 30.53717,114.3488）输入坐标，系统自动解析并设置为导航起终点。用户无需

手动输入坐标，直接单击地图上的任意位置即可将该点设为起点或终点，系统自动捕获经纬度并填充到输入框中，同时在地图上添加临时标记。

(2) 详细导航指令生成（BottomSheet）

用户启用“详细导航描述”后，路径规划成功时会自动弹出底部弹窗，显示“ 向北走约 50 步 (30.2 米)”的自然语言指令。指令基于用户身高换算步长，合并了连续相同方向的路段，避免了信息冗余。

测试结果：以两条为例，生成 5~10 条指令，软件根据每一个岔路以及方向角变化时，都会给出一个方位进行导航，步数估算合理（身高 170cm 步长约 0.765m，100 米约 130 步）。用户可通过设置关闭该功能。

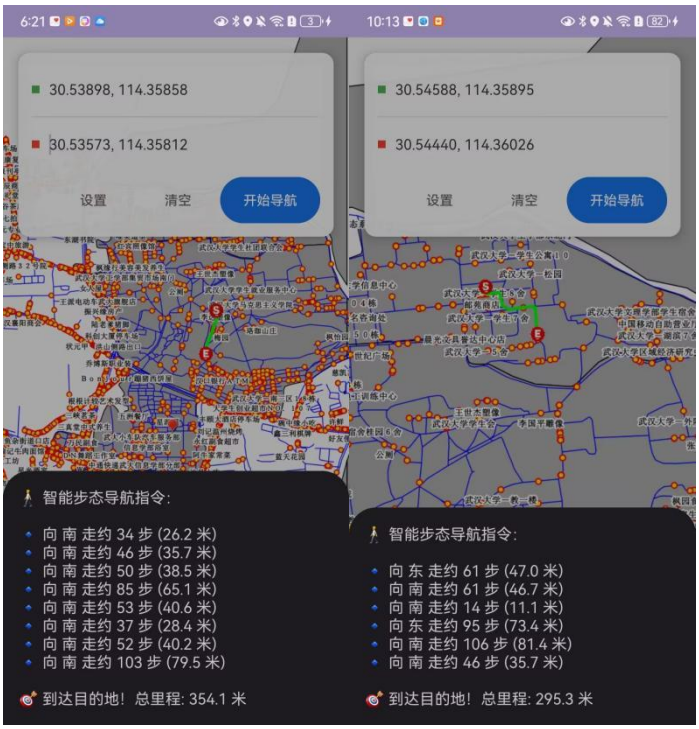


图 26 导航指令生成示例图

(3) 用户配置界面（SettingsActivity）

点击主界面“设置”按钮，跳转至独立设置页面。用户可输入身高（100~250 cm）、选择定位频率、开关详细导航描述和手动 NMEA 解析。配置保存到 SharedPreferences，返回主界面后立即生效。



图 27 设置页面

(4) 自定义图标缩放效果

原始起点/终点图标尺寸过大（约 200×200 像素），单独设计了图标，缩放至 15×15 像素后，标记大小适中，不影响地图点击和视图。

六、 总结

经过一整个学期的实验与开发，我从对导航电子地图的懵懂认知，到最终能够独立完成一款具备定位、检索、路径规划等基本功能的校园导航应用，整个过程收获颇丰，也踩了不少坑。

首先是技术层面。在实验开始前，我只在课堂上听说过 PostGIS、GeoServer 这些名词，觉得它们离实际开发很远。真正动手配置数据库、导入路网数据、发布 WFS 服务之后，我才理解了一套完整 GIS 服务链是如何运转的。其次是客户端开发。Android 的异步任务、主线程阻塞、内存泄漏这些概念，以前只在书本上见过。这次在加载 GeoJSON 数据时，我一开始直接在 UI 线程里做网络请求和 JSON 解析，结果地图拖动卡顿严重。后来改用 AsyncTask 配合线程池，再把解析好的点数据通过 Handler 回传给主线程，流畅度明显提升。另外，高频重绘 Marker 时曾经出现 OOM，排查后发现是在循环中不断新建 Bitmap 对象而没有复用。改成单例模式和软引用缓存后问题解决。这些经验对我来说非常宝贵。

还有一个感触是“产品思维”的落地。实验中有一个拓展功能是步态导航播报，需要根据用户身高步长估算行进距离，然后在底部滑动卡片里显示“前方 50 米右转”这样的文字指令。一开始我觉得算出最短路径就结束了，但真正上手做播报系统才发现，路径只是一串坐标点，要把它翻译成人能看懂的文字指令，需要判断拐弯方向、计算剩余距离、过滤冗余节点。这个过程让我理解了为什么商业导

航软件的体验差异往往不在算法精度，而在交互细节。于是做了一下两点优化：

一是页面设计的优化。实验初期的界面非常简陋——一个全屏地图控件，底部堆着两个输入框和几个按钮，虽然功能上能跑通，但用起来很生硬。参考高德地图的移动端设计规范后，我重新梳理了布局：将起终点输入框收拢到顶部悬浮卡片中，右侧增加“互换”按钮方便交换起终点；底部采用上滑抽屉（BottomSheet）展示导航指令和行程信息；定位、路径搜索、清除等功能按钮统一用悬浮圆形按钮（FAB）置于右下角，并加入了按压反馈和过渡动画。这样修改后，界面清爽了很多，操作也更符合直觉——用户不需要刻意寻找功能入口，常用操作都在拇指可控范围内。

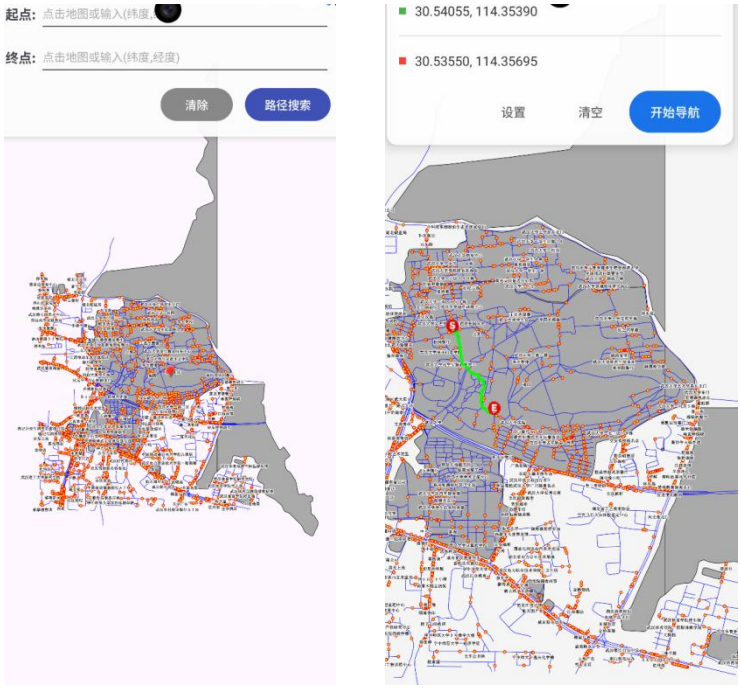


图 28 页面布局优化：优化前（左）优化后（右）



图 29 参考范例

二是标点按键的重新设计。原本地图上的 POI 标记使用的是 osmdroid 默认的红色水滴图标，在校园建筑密集的场景下，多个标记重叠时很难区分。我为起

点终点设计了图标。图标是先在绘图软件里画出 24x24 像素的矢量图，再导出为 PNG 并转为 Android 的 Drawable 资源。

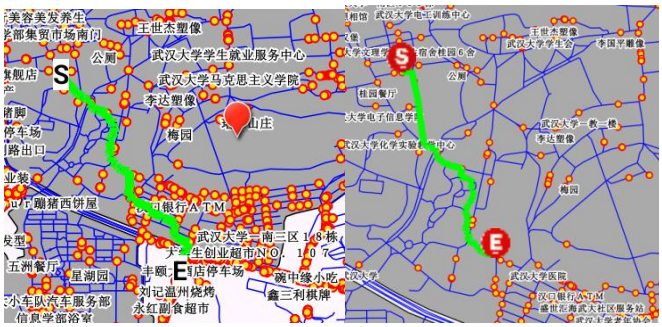


图 30 标点设计：优化前（左）优化后（右）

当然，实验过程中也留有一些遗憾。比如由于时间和设备限制，定位模块只测试了室外空旷环境，在信息学部教学楼附近的多路径效应比较明显，定位蓝点偶尔会跳变。另外，路径规划目前只支持步行模式，没有考虑自行车道和车辆禁行规则。如果后续有时间，我希望在路网权重中加入高程数据，让算法优先选择平缓路段，这对武大这种坡多的校园会更实用。

总的来说，这门课程让我从“会看地图”变成了“能造地图”。虽然最终的作品离高德、百度那样的商业软件还很远，但整个全栈开发过程让我对 GIS 工程有了实实在在的体感。未来无论是继续深造还是从事开发工作，Andriod Studio 的开发经历都会是我最重要的经历。

综合评语：

条 目	**分数（百分制）**	**比例**	**最终分数**	**备注**
课程报告		50%		
平时成绩 1		5%		
平时成绩 2		20%		
平时成绩 3		25%		
总评成绩				
指导教师： 年　　月　　日				